Tutorial: Building virtual knowledge graphs for life sciences with Ontop-VKG

Carsten Fortmann-Grote[1][0000−0002−2579−5546], Damien Goutte-Gattat[2][0000−0002−6095−8718], Tiago Lubiana[2][0000−0003−2473−2313], Joshua Moore[2][0000−0003−4028−811X], Daniel Olvera Cabrera[1][0000−0002−7935−9715], and Andra Waagmester[3,4][0000−0001−9773−4008]

[1] Max Planck Institute for Evolutionary Biology, Plön, Germany
[2] German Bioimaging – Gesellschaft für Mikroskopie und Bildanalyse e.V., Konstanz University, Konstanz, Germany
[3] Micelio, Antwerpen, Belgium
[4] AmsterdamUMC: Amsterdam, North Holland, The Netherlands

Abstract. The vast majority of open source and of commercial life science data management systems are backed by a relational (SQL) database complemented by a web frontend with filtering and search functionality and a dedicated API for programmatic access. In some cases these APIs follow established web standards, such a JSON-LD or RDF/XML, making them "in principle" consumable by semantic web applications, e.g. SPARQL queries. Where this is not the case, the W3C standard for R2RML provides the basis for an alternative solution for exposing the data as a "virtual" RDF resource. Here, we propose a half–day tutorial on the topic of virtualization using the open source R2RML implementation Ontop-VKG (https://ontop-vkg.org).
Participants will learn how to configure Ontop-VKG for various database management systems, how to write mapping definitions utilizing established life science ontologies, how to expose the mapped data in a SPARQL endpoint and how to materialize the virtual graph to an RDF file. The bioimage data management system OMERO will serve as an example. After completing this session, participants will be able to expose their public life science data deposited in e.g. genome databases, organism collections, omics data warehouses, sequence archives or image data repositories. The tutorial is targeted at data engineers, data stewards, data managers and others interested in turning isolated data silos into interoperable linked open data resources. The tutorial will be held as a lecture interleaved with live coding and hands-on exercises similar to The Carpentries workshops.

Keywords: Knowledge Graphs, Research Data Management, OMERO, FAIR Data, Database, SQL

1 Background

Semantic Web Applications play an ever increasing role in the life sciences [8,9]. Nevertheless, the majority of openly accessible biological, bioinformatical, med-

ical and other life science oriented databases employ the traditional stack of a Relational Data Base Management System (RDBMS) to host the data and an HTML web frontend facing the user. Programmatic access, if present, is realized through an Application Programming Interface (API), serving the data in either XML or JSON. Both API request and response payloads often follow a non–standard self–made schema, developed for the specific application. More rarely, the API employs the JSON-LD or another Resource Description Framework (RDF) format, enabling structured queries through semantic web technology such as SPARQL. Even then, interoperability with other RDF resources as well as SPARQL based exploration of the JSON-LD graph remains limited[5].

An alternative approach to make data in RDBMSs SPARQable is by way of knowledge graph virtualization, explained in the next subsection.

1.1 Virtual Knowledge Graph (VKG) construction

Virtualization can be understood as a translation and rewriting machinery for SPARQL queries and SQL responses [10]. The concept is visualized in Fig. 1: The

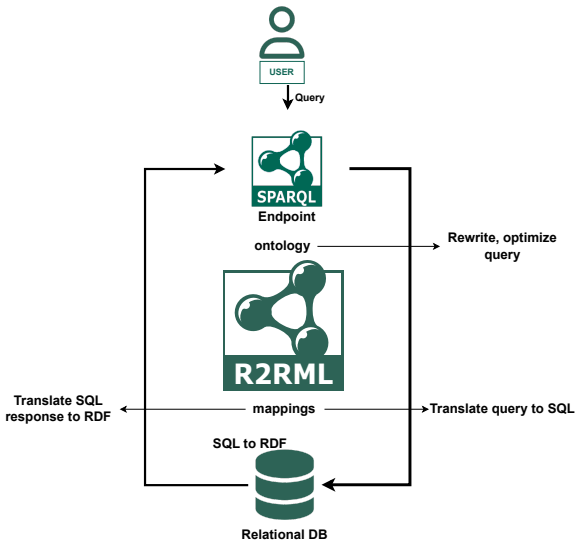


Fig. 1. Flowchart illustrating the concept of knowledge graph virtualization

submitted query is first rewritten and potentially optimized using the mapping

[5] As an example, in a single query, only IRI references and literals in the queried JSON-LD document can be accessed, objects defined in other, referenced JSON-LD documents remain invisible

definitions and the supplied ontology. The rewritten SPARQL query is then translated ("mapped") to SQL, optimized, and finally run against the database backend resulting in a query response (SQL table). The SQL response is then translated back to RDF and returned to the user as a response from the SPARQL endpoint.

In this tutorial, we employ the Ontop-VKG virtualization framework [6]. Ontop-VKG is an implementation of the W3C standard for Relational Database To RDF Mapping Language (R2RML) [1].

1.2 Setup and required software

In the following, we list required software for the tutorial. Unless otherwise noted, participants should install this software on their device if they wish to follow the life coding and hands-on exercises.

Development environment A text editor with SPARQL and RDF syntax highlighting enabled or Integrated Development Environment (IDE) is needed for editing RDF documents, OBDA mappings and SPARQL queries. Participants familiar with the ontology management software Protege[7] are advised to install the Ontop-VKG plugin available from Ref. [5] and to familiarize with the mapping editor. Note that we will not cover Protege in this tutorial. A web browser is needed to access the ontop SPARQL query frontend.

ontop-cli To install the Ontop-VKG command line utilities, navigate to the website referenced in Ref. [5], download and extract the zip file "ontop-cli-5.4.0.zip". Add the unzipped directory to your `$PATH` variable. We recommend version 5.4.0, the most recent version at the time of writing.

PostgreSQL RDBMS A RDBMS is needed to host the relational databases against which to develop R2RML mappings. Here, we use the PostgreSQL RDBMS. **Note: A PostgreSQL server will be hosted on a publicly accessible cloud server, participants only need a PostgreSQL client, such as `psql`.**

OMERO Test Infrastructure and omero-ontop-mappings Clone the github repository Ref. [2]. It provides a utility script to install, configure, and launch the OMERO test infrastructure [3].

1.3 Tutorial program

Introduction After a short welcome and introductions of the speakers, we will give a presentation of the topic of VKG construction along the lines of this paper's introduction (Sec. 1).

Warm up: Translating a CSV table to RDF We illustrate the basic idea of R2RML mappings by manually mapping a single table of comma separated values (CSV) to an RDF graph. The table lists the planets of the solar system and their physical properties[6]. Taking one planet (e.g. planet Earth) as an example, we explain how to map the table name to an RDF class, the primary key indexing the table rows to class instances, column headers to RDF properties and column values to RDF objects (literals). We then assign the remaining planets to groups of participants to work out the RDF triples for each planet in the solar system. A data resource will be provided. All triples will then be merged into one document.

Mapping the Planets Database The second part utilizes the same data as the warm-up, the table of planetary properties. This time though, the data resides in a PostgreSQL database, set up for this tutorial. We explain how to configure ontop in order to connect to the database and provide a slim ontology file and a mappings file with only minimal content such that the explanations are functional. We explain the syntax and grammar of OBDA mappings by developing a mapping which instantiates objects of class `:Planet` from the id column of the table. The participants are then again split into groups. Each group is assigned to one of the remaining columns in the planets table and to write up the corresponding mappings. All mappings will then be merged into on OBDA file. Finally, we demonstrate how to launch the ontop SPARQL endpoint and let the participants run example SPARQL queries.

In the next level we include another table "moons" from the same database. The "moons" table lists all moons of the solar system. The planet–moon relationship is established by the "planet" column in the "moon" table, containing the planet's id as a foreign key. In this way, we explain how relationships between two class instances can be established and how to express such relations in ontop via SQL's `JOIN` operator. Furthermore, we add statements to the ontology to express the semantic relationships between planets and moons.

Mapping the OMERO database Finally, we turn towards a real life biologically relevant database, OMERO. OMERO is a research data management system for bioimage data. Ontop mappings for OMERO are subject to ongoing development efforts. The mappings, ontology and cookiecutter templates to facilitate deployment for a given OMERO instance are provided in our github repository [2].

We will demonstrate how to deploy ontop for the OMERO test infrastructure, launch the SPARQL endpoint and how to materialize the OMERO VKG to an RDF file stored on disk. We conclude by giving an overview of already existing mappings and by pointing out important mappings still missing. Participants are then tasked to add a mapping of their choice or to code up SPARQL queries against the OMERO VKG.

[6] We must admit that this example is not exactly representing biological data. We choose it here because of its familiarity and simple yet non–trivial hierarchical structure

Optional: Mappings for OMOP OMOP [4] is a community driven metadata standard and SQL schema for observational studies in the life sciences. If time permits, we will task the participants to develop mappings for an example SQL database following the OMOP schema.

Acknowledgments. This work was supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under the National Research Data Infrastructure – NFDI 46/1 – 501864659 ("NFDI4BIOIMAGE").

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. R2RML: RDB to RDF Mapping Language, https://www.w3.org/TR/r2rml, [Online; accessed 8. Jun. 2022]
2. omero-ontop-mappings (2025), https://github.com/German-BioImaging/omero-ontop-mappings
3. omero-test-infra (2025), https://github.com/ome/omero-test-infra
4. Omop (2025), https://ohdsi.github.io/CommonDataModel/
5. ontop-cli (2025), https://github.com/ontop/ontop/releases/tag/ontop-5.4.0
6. Calvanese, D., et al.: Ontop: Answering sparql queries over relational databases pp. 1278–2490, http://www.semantic-web-journal.net/content/ontop-answering-sparql-queries-over-relational-databases-1
7. Noy, N.F., Crubezy, M., Fergerson, R.W., Knublauch, H., Tu, S.W., Vendetti, J., Musen, M.A.: Protégé-2000: an open-source ontology-development and knowledge-acquisition environment. **2003**, 953
8. Peyton, M.P., Chung, C.H., Mohanty, S., Truong, V.Q., Scouten, A., Shukla, S., Lazard, C., Masood, D., Murphy, J.D., Ketter, A., Ahooyi, T.M., Amin, V.R., Arora, A., Baron, J.A., Bazzi, F., Bondali, A., Bristy, N.A., Carter, E.K., Chen, Y., Courneya, J.P., Daniels, C., Domagalski, M.J., Felix, V., Ho, V.W., Holko, M., Katayama, T., Kebede, Y., Kim, M., Kini, R., Soria, C.L.D., Limdi, A., Mitra, S., Molinelli, E., Morris, E., Mulbagal, A.M., Paul, B., Pericherla, S.R., Quaid, K., Robotin, R., Rusina, P., Santiago, I.L., Stear, B., , K., Sundar, S., Surapaneni, L., Taylor, D.M., Weyand, S., Wingfield, B., Withers, C., Yuan, D.Y., Richardson, E., Rabindran, B., Busby, B.: A blueprint for open science: How transatlantic teams built and deployed knowledge graphs to enable biological (AI) models. https://doi.org/10.37044/osf.io/g4rk2_v1, https://osf.io/preprints/biohackrxiv/g4rk2_v1/
9. Wang, L., Hao, H., Yan, X., Zhou, T.H., Ryu, K.H.: From biomedical knowledge graph construction to semantic querying: a comprehensive approach **15**(1), 8523. https://doi.org/10.1038/s41598-025-93334-5, https://www.nature.com/articles/s41598-025-93334-5
10. Xiao, G., Ding, L., Cogrel, B., Calvanese, D.: Virtual Knowledge Graphs: An Overview of Systems and Use Cases. Data Intelligence **1**(3), 201–223. https://doi.org/10.1162/dint_a_00011